

```
на csv_path = os.path.join(base_dir, "../../experiments/data/lin_400_train.csv")
```

с параметрами:

```
time_steps = 10
```

```
count_size=10, # количество слоев
```

```
gru_units=100, # количество юнитов (нейронов) в каждом слое GRU
```

по MSE получаем:

```
Average time per epoch: 0.86 seconds
```

```
Loss before training = 0.35972481966018677
```

```
Loss after training = 0.0013143420219421387
```

```
На csv_path = os.path.join(base_dir, "../../experiments/data/ode_train_1000.csv")
```

с такими же параметрами имеем:

```
Average time per epoch: 1.88 seconds
```

```
Loss before training = 6.45160436630249
```

```
Loss after training = 2.1396543979644775
```

```
different between Losses =4.311949968338013
```

Давайте сравним лосс на валидации и обучении, чтобы извлечь из этого выводы и уменьшить потерю на обучении:

нужно:

Передать данные в модель в качестве валидационной выборки во время обучения.

Оценивать лосс на валидации, чтобы понимать, происходит ли переобучение.

```
gru_model.train(train_data_x, train_data_y, validation_data=(val_data_x, val_data_y), epochs=50,  
verbose=0, callbacks=[time_callback])
```

сравним на примере датасета ode_train_400.csv:

```
Average time per epoch: 1.17 seconds
```

```
Loss before training = 2.1407418251037598
```

```
Loss after training = 2.133284568786621
```

```
different between Losses =0.007457256317138672
```

```
validation loss before training = 1.956415057182312
```

```
validation loss after training = 1.9611810445785522
```

```
Difference in validation loss = -0.004765987396240234
```

какие выводы?

Разница между потерями до и после обучения незначительна (0.0075), что может говорить о том, что модель почти не улучшилась (не обучилась) на обучающей выборке за 50 эпох обучения

опираясь на данные валидации:

loss на валидации даже немного увеличился после обучения. Это может указывать на то, что модель: Начинает переобучаться: Она учится слишком хорошо подстраиваться под обучающие данные, но её способность обобщать на новых данных (валидация) ухудшается.

Попробуем увеличить значение epochs до =100 и посмотрим на результат:

```
Average time per epoch: 0.92 seconds
```

```
Loss before training = 7.131373405456543
```

```
Loss after training = 2.155355930328369
```

```
different between Losses =4.976017475128174
```

```
validation loss before training = 6.62426233291626
```

validation loss after training = 1.9661611318588257

Difference in validation loss = 4.658101201057434

Видим, что лосс на тренировочных данных уменьшился, что хорошо. Лосс на валидации примерно сопоставим с тренировочными - говорит о том, что модель хорошо обучается (и переобучения в ней нет)

Попробуем увеличить количество эпох до 200:

Average time per epoch: 0.78 seconds

Loss before training = 2.16206431388855

Loss after training = 2.131352186203003

different between Losses = 0.030712127685546875

validation loss before training = 1.9626861810684204

validation loss after training = 1.9657098054885864

Difference in validation loss = -0.0030236244201660156

второй запуск: ????

Average time per epoch: 1.03 seconds

Loss before training = 8.405294418334961

Loss after training = 2.1324706077575684

different between Losses = 6.272823810577393

validation loss before training = 7.84114933013916

validation loss after training = 1.961984395980835

Difference in validation loss = 5.879164934158325

Заметим, что результаты от запуска к запуску отличаются, причем очень сильно. Это происходит из-за того, что начальные веса в tf определяются рандомно и для какого-то запуска они могут быть подобраны хорошо, а для другого плохо. Поэтому зафиксируем seed=42 и обучим модель еще раз на количество эпох=200:

для этого изменим немного топологию:

seed = 42

random.seed(seed)

np.random.seed(seed)

tf.random.set_seed(seed)

В классе инициализируем веса так:

weight=keras.initializers.RandomUniform(minval=-1, maxval=1,seed=seed),

biases=keras.initializers.RandomUniform(minval=-1, maxval=1,seed=seed),

Результат:

Total training time: 214.47460842132568 seconds

Average time per epoch: 1.0723730421066284 seconds

Loss before training = 62.21070098876953

Loss after training = 2.1394999027252197

different between Losses = 60.07120108604431

validation loss before training = 60.87479019165039

validation loss after training = 1.9602504968643188

Difference in validation loss = 58.91453969478607

при повторных запусках обучения результаты лосс функции очень схожи, а следовательно теперь мы

можем объективно оценить обучение нейронной сети:

лосс валидации и обучения примерно равны. Лосс по валидации до обучения и после обучения значительно снизился, что говорит о хорошем обучении нс.

(примерно такие же результаты модель дает на эпохах=50)

Epoch 50 finished in 1.07 seconds

Total training time: 57.28201127052307 seconds

Average time per epoch: 1.1456402254104614 seconds

Loss before training = 62.21070098876953

Loss after training = 2.1385626792907715

different between Losses = 60.07213830947876

validation loss before training = 60.87479019165039

validation loss after training = 1.959972858428955

Difference in validation loss = 58.914817333221436

25.10.24

1. Написал функцию $\text{hardsin} = \sin(\ln(x^{\sin(x)}))$
2. поправил датасет, для генерации последовательных точек
3. Увеличил модель для большей точности:
 - a. эпохи=250
 - b. слои=10
 - c. нейроны=100

для **hardsin** получил такие результаты:

ТЕСТ №1

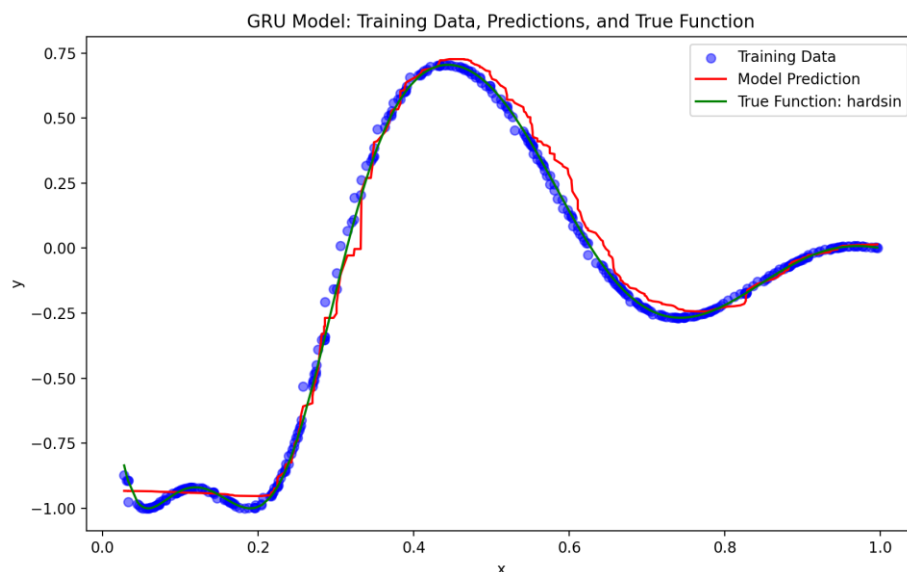
Total training time: 242.56444430351257 seconds

Average time per epoch: 0.9702577772140503 seconds

Loss before training = 24.74151611328125

Loss after training = 0.0025757704861462116

different between Losses = 24.738940342795104



validation loss before training = 24.862573623657227

validation loss after training = 0.017788385972380638

Difference in validation loss = 24.844785237684846

ТЕСТ №2

- Добавим график изменения лосс функций на обучающейся и валидационной выборке.

Если на графике видно, что **лосс на обучающей** выборке продолжает **снижаться**, а **лосс на валидационной** выборке начинает **расти** после определённого количества эпох, это указывает на переобучение

эпохи=200

слои=10

нейроны=100

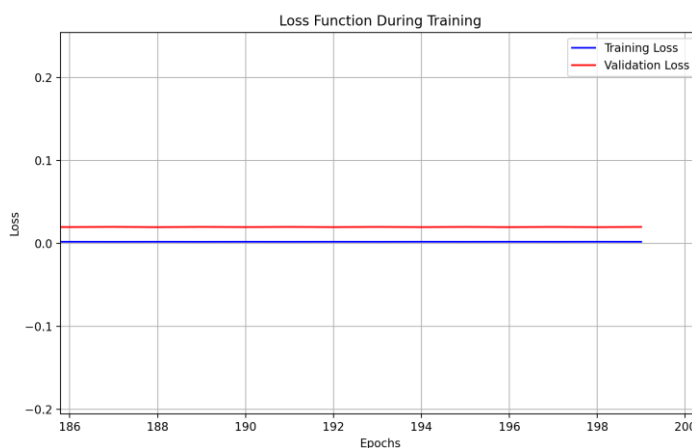
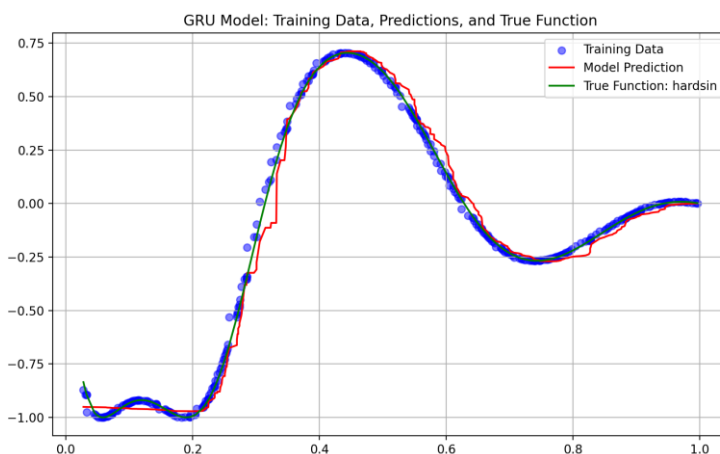
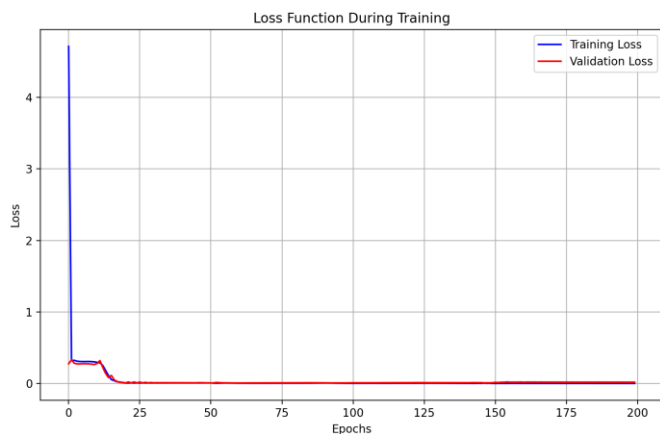
Total training time: 188.4122829437256 seconds

Average time per epoch: 0.9420614147186279 seconds

Loss before training = 24.74151611328125

Loss after training = 0.0029667664784938097

different between Losses =24.738549346802756



validation loss before training = 24.862573623657227

validation loss after training = 0.019802477210760117

Difference in validation loss = 24.842771146446466

Видим, что валидация, пусть и незначительно, но растет, по сравнению с обучающей выборкой. Отсюда делаем вывод, что создание дропаута может улучшить поведение модели.

Попробуем добавить шум в датасет, чтобы оценить как модель будет угадывать значения:

```
mu = np.mean(nn_data_y)
sigma = np.std(nn_data_y)
noise = np.random.normal(mu, sigma, nn_data_y.shape)
nn_data_y_noisy = nn_data_y + noise
```

Результат:

ТЕСТ №3

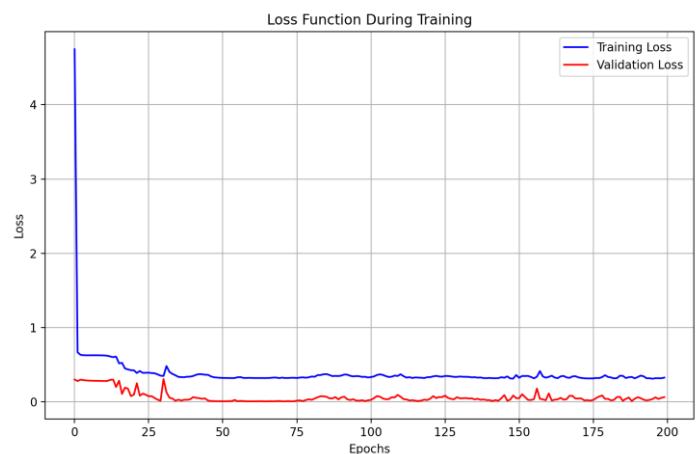
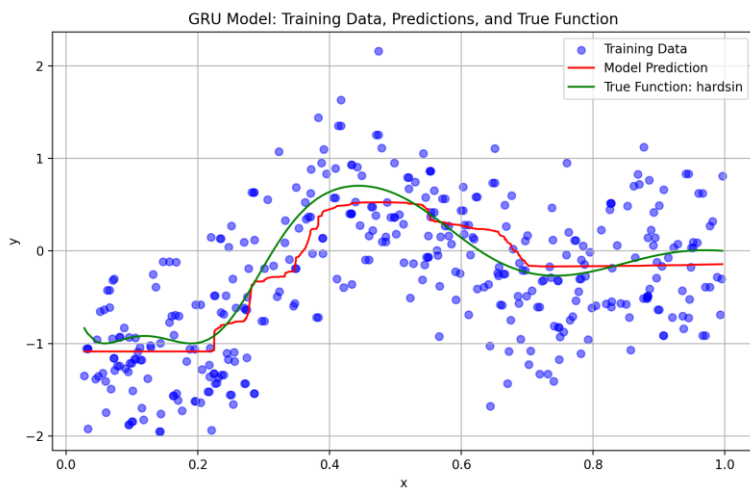
Total training time: 147.8683581352234 seconds

Average time per epoch: 0.7393417906761169 seconds

Loss before training = 23.784385681152344

Loss after training = 0.31305909156799316

different between Losses = 23.47132658958435



validation loss before training = 24.862573623657227

validation loss after training = 0.06501126289367676

Difference in validation loss = 24.79756236076355

Видим, что модель достаточно неплохо справилась с задачей при наличии шумов. Однако, аппроксимация не столь точная, как хотелось бы.

Что можно сделать для улучшения точности при шумных данных?

- **Увеличение количества слоев или юнитов GRU:** Более сложная архитектура
- **Регуляризация:** L2-регуляризация в слоях GRU может помочь снизить влияние шума, добавив штраф за слишком большие веса, которые могут быть адаптированы к шуму.

Попробуем увеличить количество эпох=500 и посмотрим на результат:

ТЕСТ №4

count_size=10, # количество слоев

gru_units=100, # количество юнитов (нейронов) в каждом слое GRU

Total training time: 373.3819422721863 seconds

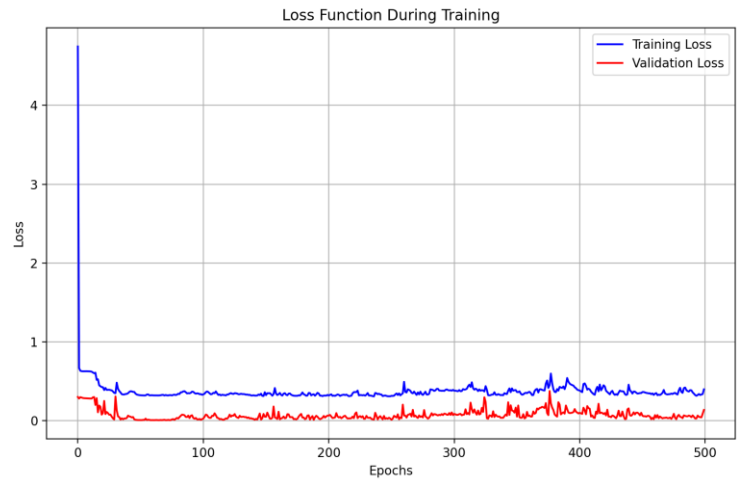
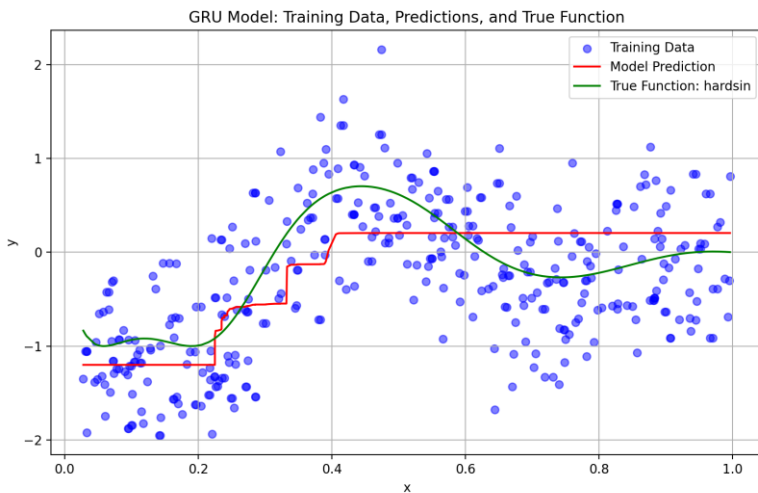
Average time per epoch: 0.7467638845443726 seconds

Loss before training = 23.784385681152344

Loss after training = 0.4023158848285675

different between Losses = 23.382069796323776

validation loss before training = 24.862573623657227



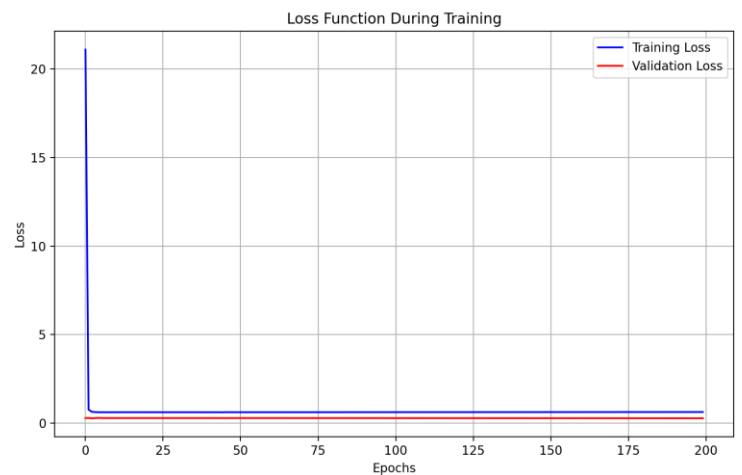
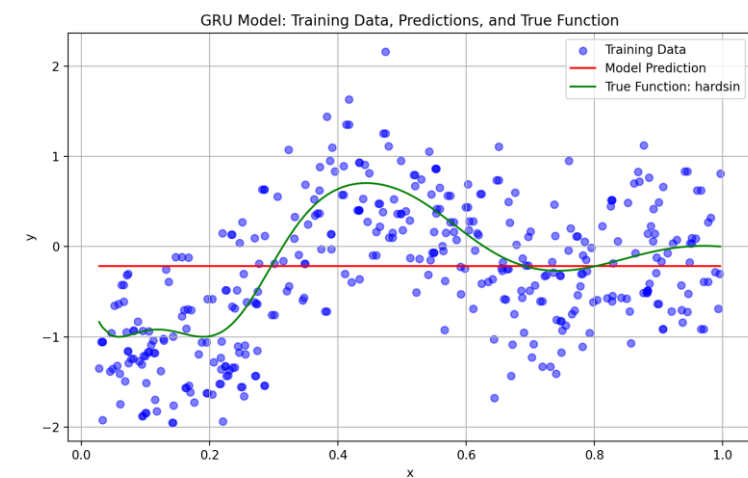
validation loss after training = 0.1378963738679886

Difference in validation loss = 24.724677249789238

Видим, что модель не переобучается, но точность аппроксимации стала хуже. Давайте попробуем **усложнить модель:**

ТЕСТ №5

count_size=20, # количество слоев

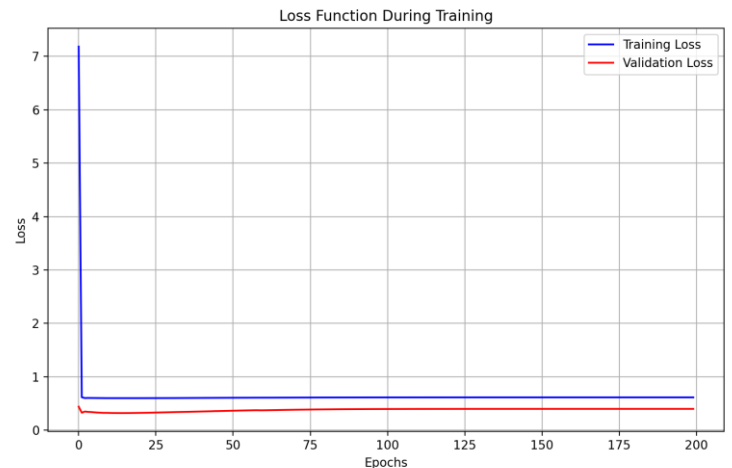
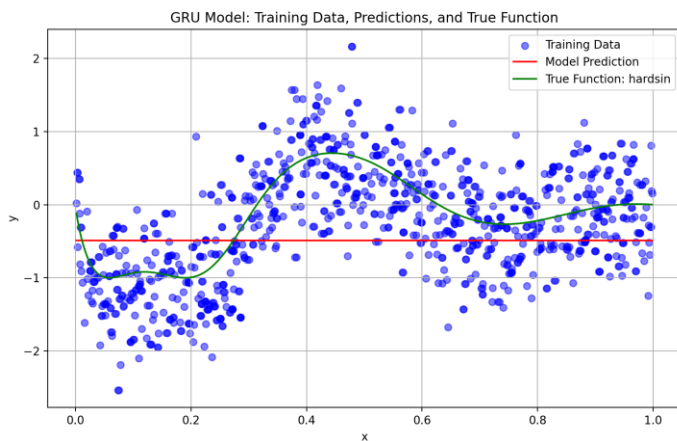


gru_units=200, # количество юнитов (нейронов) в каждом слое GRU
epochs=200

видим какую-то жесть. Модель не успевает обучиться, лосс обучения сразу падает на лосс валидации. Модель слишком быстро обучается. Попробуем уменьшить с **time_steps = 10** до **=3**, чтобы создать больше пар точек для предсказания, а так же увеличим датасет до 1000 точек:

ТЕСТ №6

Total training time: 491.0201711654663 seconds
Average time per epoch: 2.4551008558273315 seconds
Loss before training = 1.799167275428772
Loss after training = 0.6360214352607727
different between Losses = 1.1631458401679993
validation loss before training = 1.2303135395050049
validation loss after training = 0.39819812774658203
Difference in validation loss = 0.8321154117584229

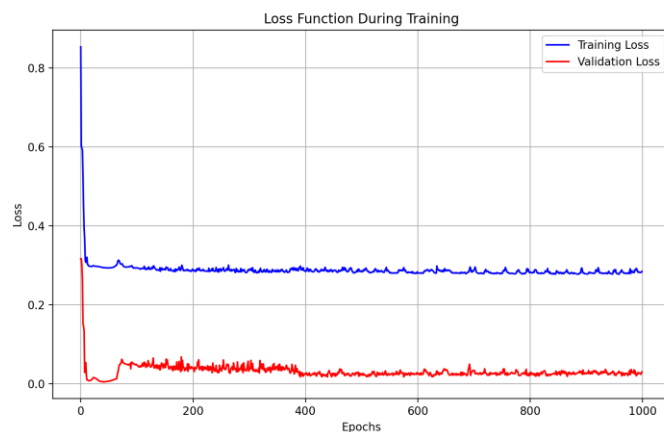
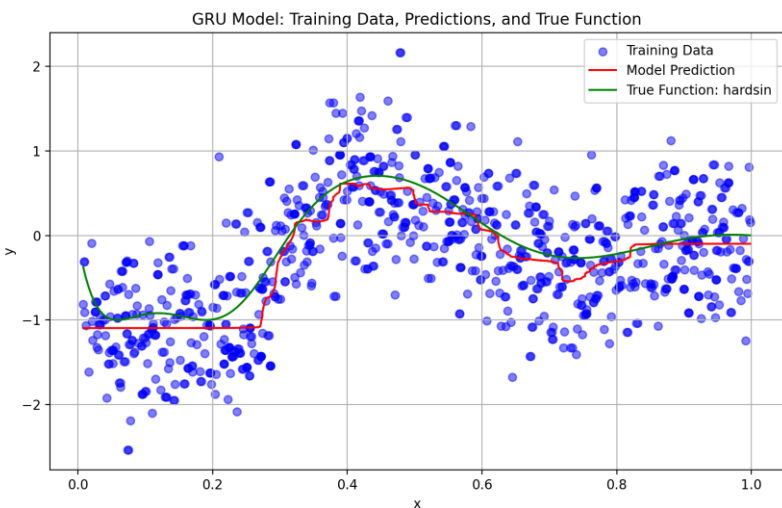


хз че с этим делать, запустим с

ТЕСТ №7

epochs=1000
count_size=5, # количество слоев
gru_units=50, # количество юнитов (нейронов) в каждом слое GRU
time_steps = 10

Total training time: 730.9099929332733 seconds
Average time per epoch: 0.7309099929332733 seconds
Loss before training = 2.816875696182251
Loss after training = 0.2755383849143982
different between Losses = 2.541337311267853
validation loss before training = 2.105607509613037
validation loss after training = 0.03061738610267639
Difference in validation loss = 2.0749901235103607



ТЕСТ №8

epochs=1000

count_size=10, # количество слоев

gru_units=100, # количество юнитов (нейронов) в каждом слое GRU

time_steps = 10

Total training time: 1577.5595259666443 seconds

Average time per epoch: 1.5775595259666444 seconds

Loss before training = 24.081575393676758

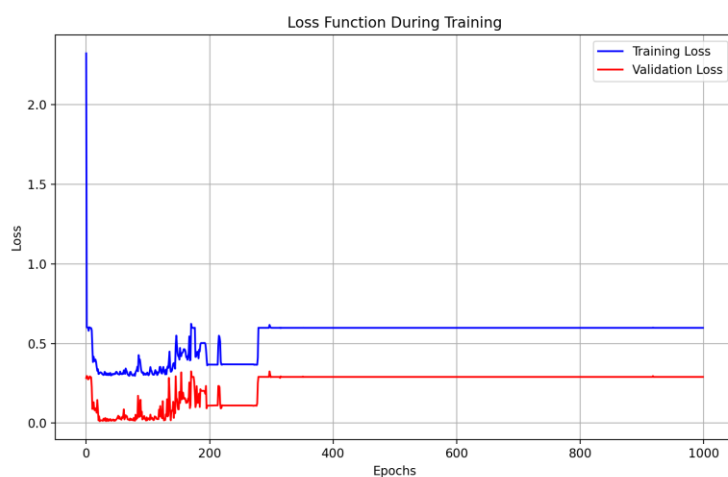
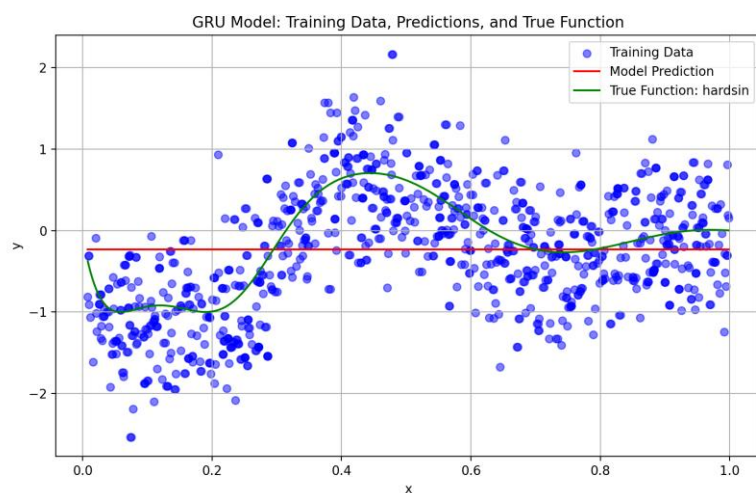
Loss after training = 0.5968294143676758

different between Losses = 23.484745979309082

validation loss before training = 25.15126609802246

validation loss after training = 0.28979918360710144

Difference in validation loss = 24.86146691441536



Таким образом у нас предсказания опять выстроились в прямую.

Уменьшим эпохи и слои и нейроны в слоях

ТЕСТ №9

epochs=100

count_size=5, # количество слоев

gru_units=20, # количество юнитов (нейронов) в каждом слое GRU

time_steps = 10

Total training time: 68.04035782814026 seconds

Average time per epoch: 0.6804035782814026 seconds

Loss before training = 0.8097265362739563

Loss after training = 0.2840789556503296

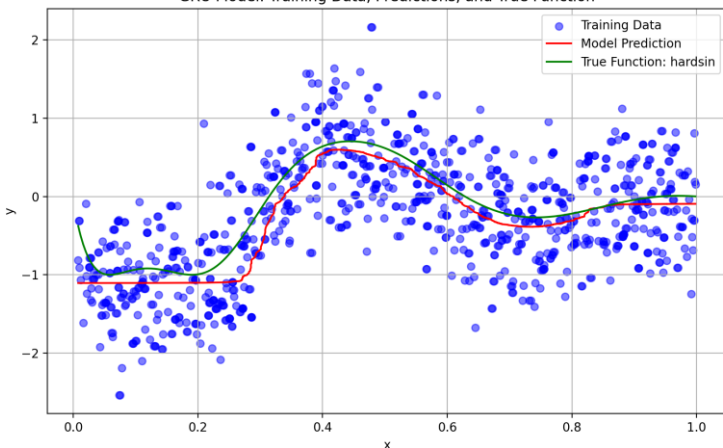
different between Losses =0.5256475806236267

validation loss before training = 0.38612237572669983

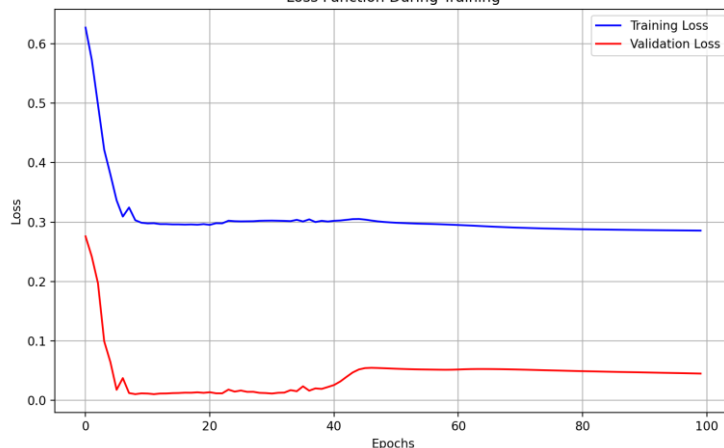
validation loss after training = 0.04508964717388153

Difference in validation loss = 0.3410327285528183

GRU Model: Training Data, Predictions, and True Function



Loss Function During Training



ТЕСТ №10

Попробуем убрать шум с данных и обучить модель на таких параметрах

epochs=200

count_size=20, # количество слоев

gru_units=200, # количество юнитов (нейронов) в каждом слое GRU

Почему именно такие параметры?

Потому что в тесте номер 5 с этими параметрами, но с шумом у нас была прямая- предсказания модели.

Результаты:

Total training time: 1395.0246262550354 seconds

Average time per epoch: 6.975123131275177 seconds

Loss before training = 2.5232536792755127

Loss after training = 0.30500328540802

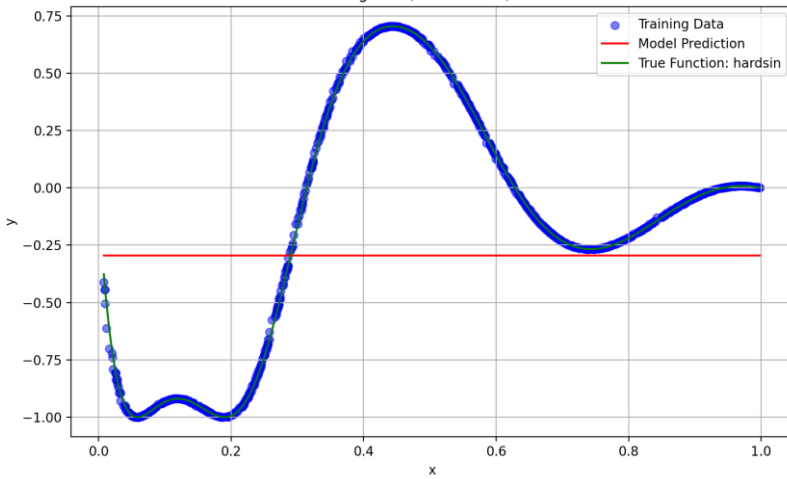
different between Losses =2.2182503938674927

validation loss before training = 2.4584367275238037

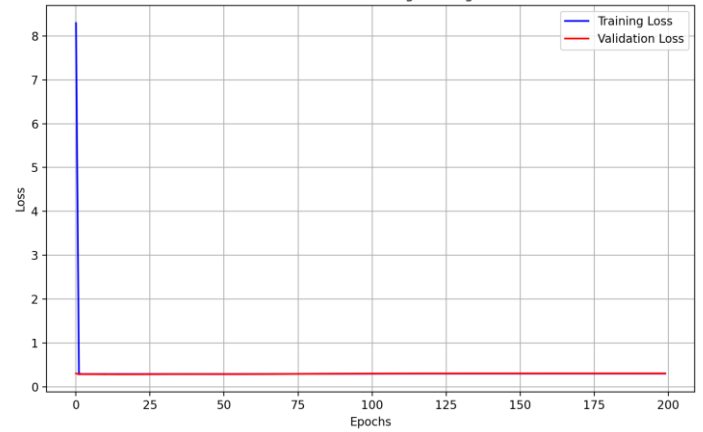
validation loss after training = 0.3051421046257019

Difference in validation loss = 2.153294622898102

GRU Model: Training Data, Predictions, and True Function



Loss Function During Training



ТЕСТ №11

Попробуем запустить этот же тест но с **optimizer="SGD"**,

epochs=200

count_size=20, # количество слоев

gru_units=200, # количество юнитов (нейронов) в каждом слое GRU

Total training time: 1537.880167722702 seconds

Average time per epoch: 7.68940083861351 seconds

Loss before training = 2.5232536792755127

Loss after training = 0.0028334816452115774

different between Losses = 2.520420197630301

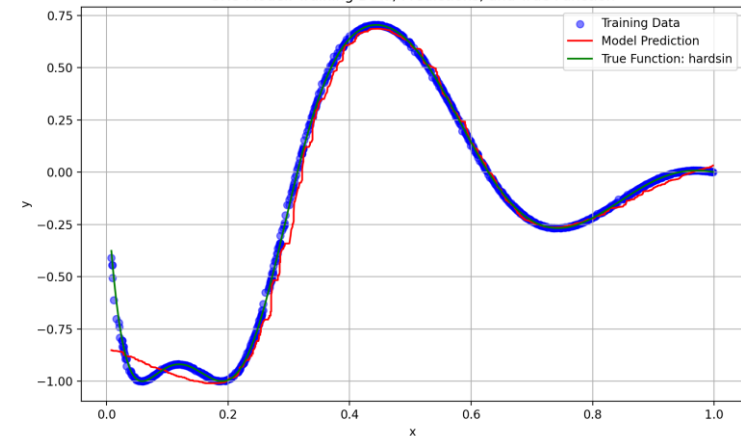
validation loss before training = 2.4584362506866455

validation loss after training = 0.006159005220979452

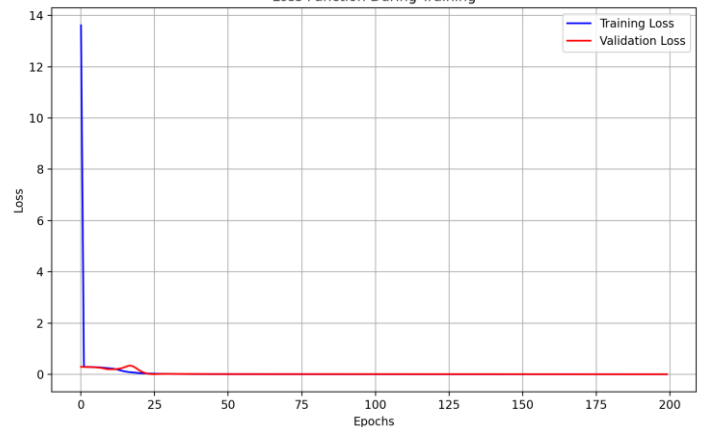
Difference in validation loss = 2.45227724546566

Время обучения: 25 минут и 13.86 секунд

GRU Model: Training Data, Predictions, and True Function



Loss Function During Training



Loss Function During Training

